

Plugin Architectures with Rust

an Analysis of Obstacles
and Prospects

20.05.2026



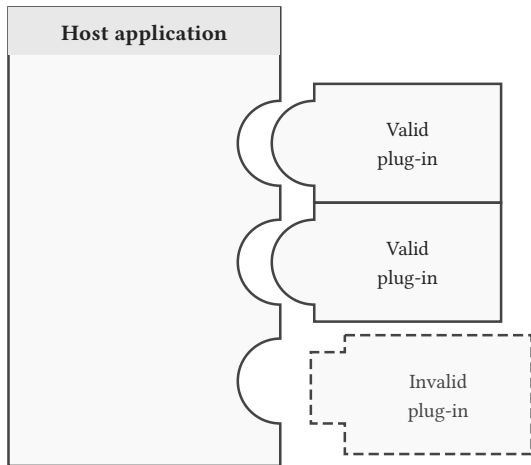
THU
Technische
Hochschule
Ulm

Contents



1	Introduction	3
2	ABI	7
3	IPC	17
4	Conclusion	22
5	Appendix	24

1 Introduction



- **Extensibility:** The ability to add new functionality post-deployment.
- **Modularity:** Plugins are isolated units that interact with the host through a *well-defined interface*.
- **Dynamic Loading:** Plugins are typically loaded at runtime rather than linked statically at compile time.

Performance Overhead

Resulting from the required overhead, how slow is the execution of the plugin compared to the rust native solution.

This is measured by using a average data fusion with 1.000.000 data points and expressed with **percentages**.

Development Complexity

- Learning curve of the implementation
- Quality of the Documentation and examples
- Required amount of work and consequences for setup, configurations and integration

This is expressed from -- **to** ++.

Criteria (ii)



(Language) Limitations

How much does this approach limit the usage of the rust languages features and ecosystem.

This is expressed from -- **to 0**.

Interoperability

How well can this approach work with other libraries, tools and especially languages.

This is expressed from **0 to ++**.

2 ABI

2.a.a Application Binary Interface

What an ABI Governs



Data Layout: struct { u8, u32, u8 }



⚠ The compiler may reorder fields — a C caller reading offset 8 would get garbage

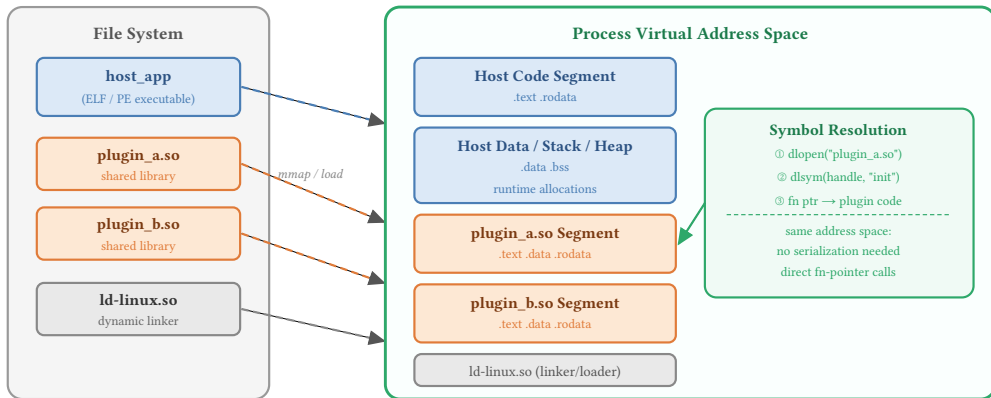
Related term: **Foreign Function Interface**

Basics (ii)



2.a.b Dynamic Linking/Loading

- 1) Discovery
- 2) Loading
- 3) Resolution
- 4) Execution





2.b.a Demonstration of dynamic linking and loading

```
// In base library (used by application and plugin):  
trait Fuser {  
    fn fuse(&self, data: &[SensorData]) -> Result<SensorData, Box<dyn Error>>;  
}
```

```
// In plugin (library):  
struct AveragePlugin;  
impl Fuser for AveragePlugin { ... }  
  
#[unsafe(no_mangle)] // ensure name can be found in binary  
pub fn average_plugin() -> Box<dyn Fuser> {  
    Box::new(AveragePlugin::new())  
}
```

(Unstable) Rust ABI (ii)



```
// In application:
type FuseFunc = extern "Rust" fn() -> Box<dyn Fuser>; // use Rust ABI
let plugin: Plugin<FuseFunc> = Plugin::new(
    &plugin_path.into(),
    "average_plugin".as_bytes()
).unwrap(); // 1), 2) and 3)

let fuser: Box<dyn Fuser> = plugin.get();
let sd = [ SensorData::new(24.0, 0.01), ... ];
let res = fuser.fuse(&sd); // 4) Execution
println!("{:?}", res);
```

(Unstable) Rust ABI (iii)



```
// In application:
struct Plugin<T> where T: Fn() -> Box<dyn Fuser> + Copy{
    lib: Library,
    func: T, // or Vec<T> or HashMap<String, T>
}
```

Using the "libloading" crate:

- 1) Discovery
- 2) Loading
- 3) Resolution
- 4) Execution

```
fn new(path: &PathBuf, s: &[u8]) -> Result<Self, libloading::Error> {
    let lib = unsafe { Library::new(path) }?; // 1+2
    let symbol: Symbol<T> = unsafe { lib.get(s) }?; // 3)
    let func: T = *symbol; // fn deref(&self) -> &T
    Ok(Plugin { lib, func })
}
fn get(&self) -> Box<dyn Fuser> { (self.func)() }
```



2.b.b but it's Unstable

The implementation itself is not complex, but...

- Compiler version changes **may** impact the generated binary
- Changes to structs, traits and enums **may** impact the generated binary
- Base library, application and plugins need to run on the **same compiler version**

But we can use the **C ABI** (next slides)! But representing Rust in C is very difficult and will introduce trade-offs.

- How are structs, traits and enums represented?
- What happens to VTables?



- Usage of `#[sabi_trait]` attribute macro for creating FFI-safe trait objects
- Allows for a lot of customization when specifying the FFI
- Provides ABI-stable alternatives to many *std-types*, but also some *external_types*
- Widely used crate with a lot of community support

but:

- Many hidden structs and types are generated and are required for further use, therefore it is overwhelming at first
- Usage of `async` is not possible, because a FFI-safe wrapper for `Future` is not supported.



- Easy to get started as primarily `#[stabby::stabby]`, `#[stabby::export]` and `stabby::dynptr!(...)` are used.
- Good explanation of the concepts behind ABI development (and why Rust doesn't like the C ABI and vice versa)
- Provides ABI-stable alternatives to common types
- Built for performance
- “Compiler-change-proof ABI-stability is proven statically through the type system.”
- Imported/Exported binaries can be checked for validity

but:

- relatively new and therefore not a lot of community engagement (yet)
- had some breaking changes between versions due to a rust compiler update
- documentation can be spotty at times

Comparison ABI crates

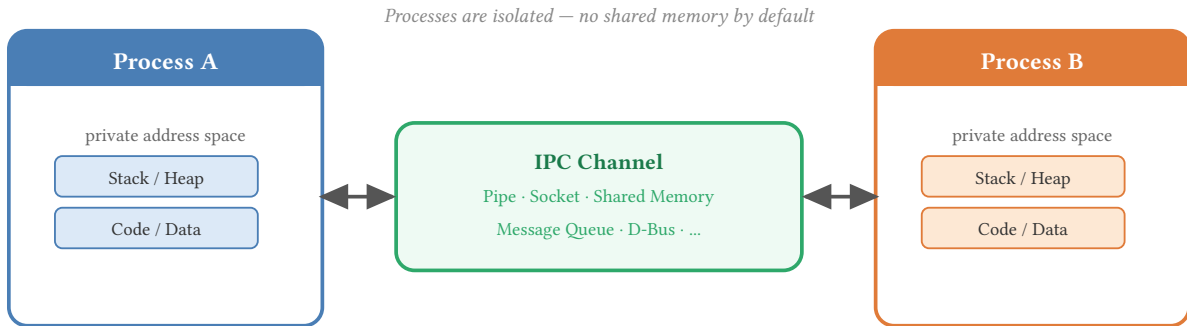


Crate	Perf. Diff	Development Complexity	Limitation	Interoperability
native		++	0	0
unstable_abi	16%	--	--	0
abi_stable	25%	0	0	+
stabby	9%	0	0	+

Table 1: Summary of ABI results.

3 IPC

Interprocess communication



Biggest takeaway: Shared memory is not accessible for this approach. Therefore (de-) **serialization** is required.

rustbridge crate



High-level JSON, native Rust speed — Work with serde types, not raw pointers!

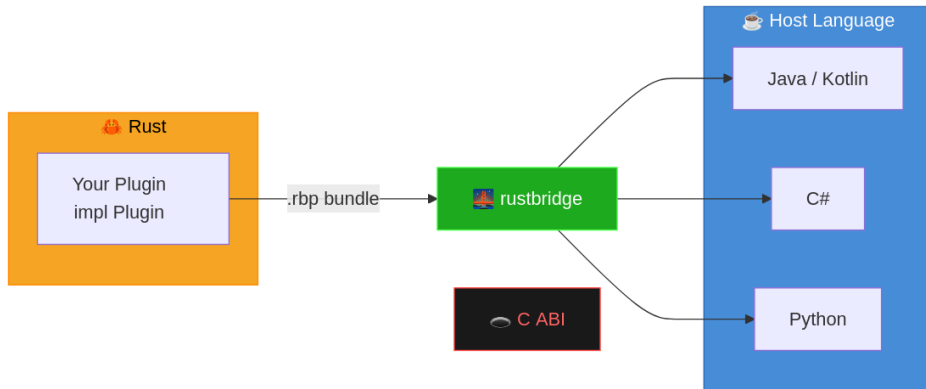


Figure 5: A rustbridge plugin can be bundled into a .rbp file and used across different languages.



- Stable C ABI — Plugins work regardless of your Rust compiler version or optimization flags
- Managed lifecycle: Startup, shutdown, including logging callbacks
- Excellent technical documentation and examples
- Communication with Request/Response Headers (JSON or Binary)
- Usage of CLI tools to create the bundle

but:

- No shared memory, therefore performance loss due to serialization
- Binary approach requires manual conversion of bytes to types
- Requires strict separation of plugin and application

Crate	Perf. Diff	Development Complexity	Limitation	Interoperability
native		++	0	0
rustbridge (JSON)	537%	+	0	++
rustbridge (Binary)	25%	-	0	++

Table 2: Summary of IPC results.

4 Conclusion

Conclusion



Crate	Perf. Diff	Development Complexity	Limitation	Interoperability
native		++	0	0
unstable_abi	16%	--	--	0
abi_stable	25%	0	0	+
stabby	9%	0	0	+
rustbridge (JSON)	537%	+	0	++
rustbridge (Binary)	25%	-	0	++

Table 3: Summary of results.

5 Appendix



5.a.a Base Library

```
#[sabi_trait] // converts this to: Fuser_T0<'lt, Pointer<()>> (and more...)
pub trait Fuser {
    #[sabi(last_prefix_field)]
    fn fuse<'a>(&self, data: RSlice<'a, SensorData>) -> RResult<SensorData, RBoxError>;
}
pub type FuserBox = Fuser_T0<'static, RBox<()>>;
```

```
#[repr(C)]
#[derive(StableAbi)]
#[sabi(kind(Prefix(prefix_ref = FuserLibRef)))]
#[sabi(missing_field(panic))]
pub struct FuserLib{
    pub new_fuser: extern "C" fn() -> FuserBox,
}
```

abi_stable implementation (ii)



```
impl RootModule for FuserLibRef {  
    abi_stable::declare_root_module_statics! {FuserLibRef}  
    const BASE_NAME: &'static str = "fuser_lib";  
    const NAME: &'static str = "fuser_lib";  
    const VERSION_STRINGS: VersionStrings = package_version_strings!();  
}
```

```
pub fn load_root_module_in_directory(directory: &Path) -> Result<FuserLibRef,  
LibraryError> {  
    FuserLibRef::load_from_directory(directory)  
}
```



5.a.b Plugin

```
// in plugin:
#[export_root_module]
fn average_plugin() -> FuserLibRef {
    FuserLib {new_fuser: average_plugin_impl}.leak_into_prefix()
}
#[sabi_extern_fn]
fn average_plugin_impl() -> FuserBox {
    Fuser_T0::from_value(AveragePlugin::new(),
                        TD_Opaque, // disallow DOWNCASTING of trait objects
    )
}
```



5.a.c Application

```
let path : PathBuf = plugin_path.into();
let data = [ SensorData::new(24.0, 0.01), ... ];
let data = RSlice::from_slice(&data);

let plugin: FuserLibRef = load_root_module_in_directory(&path)?;
let fuser: Fuser_T0<'_, RBox<()>> = plugin.new_fuser>();
let res= plugin.fuse(data);
println!("a) {:?}", res);
```



5.b.a Base Library

```
#[stabby::stabby(chcked)] // neccessary for stabby!
pub trait Fuser {
    extern "C" fn fuse(
        &self,
        data: stabby::slice::Slice<'_, SensorData>,
    ) -> stabby::result::Result<SensorData, error::Error>;
}
```

stabby implementation (ii)



5.b.b Plugin

```
#[stabby::stabby]
pub struct AveragePlugin { }
impl Fuser for AveragePlugin { ... }

#[stabby::export]
pub extern "C" fn average_plugin()
    -> stabby::dynptr!(Box<dyn Fuser>) {
    Box::new(AveragePlugin::new()).into()
}
```



5.b.c Application

```
pub type FuseFunc = extern "C" fn() -> FuserPluginDyn;
pub type FuserPluginDyn = stabby::dynptr!(stabby::boxed::Box<dyn
app_core::fuse::Fuser>);

pub struct Plugin<TDyn> {
    // Keep the plugin value before the library handle so it is dropped first.
    func: TDyn,
    _lib: Library,
}

impl<TDyn> Plugin<TDyn> {
    pub fn new<TFunc>(
        plugin_name: &PathBuf,
        symbol: &[u8],
    ) -> Result<Self, Box<dyn Error + Send + Sync>>
```

stabby implementation (iv)



```
where TFunc: PluginFactory<Dyn = TDyn> {  
    let lib = unsafe { libloading::Library::new(plugin_name)? };  
    let my_imported_function = unsafe { lib.get_stabbied::<TFunc>(symbol)? };  
    let fund = *my_imported_function;  
    let tdyn = fund.get();  
    Ok(Self { func: tdyn, _lib: lib, })  
}  
pub fn get(&self) -> &TDyn { &self.func }  
pub fn get_mut(&mut self) -> &mut TDyn { &mut self.func }  
}
```

```
let mut plugin: Plugin<FuserPluginDyn> =  
    get_plugin::<FuseFunc>(plugin_path.into(), "average_plugin".as_bytes()).unwrap();  
let data = [ SensorData::new(24.0, 0.01), ... ];  
let data: Slice<'_, SensorData> = Slice::new(&data);  
let fuser = plugin.get();
```

stabby implementation (v)



```
let res = fuser.fuse(data);

pub fn get_plugin<TFunc>(
    plugin_name: PathBuf,
    symbol: &[u8],
) -> Result<Plugin<TFunc::Dyn>, Box<dyn Error + Send + Sync>>
where TFunc: PluginFactory {
    Plugin::<TFunc::Dyn>::new::<TFunc>(&plugin_name, symbol)
}
```



5.c.a Plugin (JSON)

```
#[derive(Debug, Clone, Serialize, Deserialize, Message)]
#[message(tag = "fuse")]
struct FuseRequest { sd: Vec<SensorData>, }

#[derive(Debug, Clone, Serialize, Deserialize)]
struct FuseResponse { sd: SensorData, }

#[derive(Default)]
struct FuserPlugin;
impl FuserPlugin {
    fn new() -> Self { Self }
    fn handle_fuse(&self, req: FuseRequest) -> PluginResult<FuseResponse> {
        let fused_data = average_fuse(&req.sd); // average_fuse is the actual "execution"
of the fusion
        Ok(FuseResponse { sd: fused_data })
    }
}
```

rustbridge implementation (ii)



```
}  
}  
  
#[async_trait]  
impl Plugin for FuserPlugin {  
    async fn on_start(&self, _ctx: &PluginContext) -> PluginResult<()> {  
        register_binary_handlers();  
        Ok(())  
    }  
    async fn handle_request(  
        &self,  
        _ctx: &PluginContext,  
        type_tag: &str,  
        payload: &[u8],  
    ) -> PluginResult<Vec<u8>> {  
        match type_tag {  
            "fuse" => {
```



```
        let req: FuseRequest = serde_json::from_slice(payload)?;
        let resp = self.handle_fuse(req)?;
        Ok(serde_json::to_vec(&resp)?)
    }
    _ => Err(PluginError::UnknownMessageType(type_tag.to_string())),
}
}
}
async fn on_stop(&self, _ctx: &PluginContext) -> PluginResult<()> {
    tracing::info!("fuser-plugin stopped");
    Ok(())
}
}

fn register_binary_handlers() {
    // from rustbridge:
    register_binary_handler(MSG_FUSER_CREATE, handle_fuser_create);
}
```



```
rustbridge_entry!(FuserPlugin::new);  
// Re-export FFI functions for the shared library  
pub use rustbridge::ffi_exports::*;
```

5.c.b Application (JSON)

```
let plugin = NativePluginLoader::load_bundle(BUNDLE_PATH)?;  
if !plugin.has_binary_transport() {  
    return Err(ConsumerError::MissingSymbol(  
        "Binary transport not available in this plugin".into(),  
    ));  
}  
let sd: Vec<SensorData> = vec![SensorData::new(22.5, 0.3), SensorData::new(23.0, 0.2)];  
let response: JsonFuseResponse = plugin.call_typed("fuse", &JsonFuseRequest { sd:
```



```
sd })?;  
println!("JSON Response: {:#?}", response.sd,);
```

5.c.c Plugin (Binary)

```
pub const MSG_FUSER_CREATE: u32 = 100; // Message ID for fuser creation  
#[repr(C)]  
#[derive(Debug, Clone, Copy)]  
pub struct FuserRequestHeader {  
    pub version: u8,  
    pub _reserved: [u8; 3],  
    pub payload_size: u32,  
}  
impl FuserRequestHeader {  
    pub const VERSION: u8 = 1;
```

rustbridge implementation (vi)



```
pub const SIZE: usize = 8;
}
```

```
#[repr(C)]
#[derive(Debug, Clone, Copy)]
pub struct FuserResponseHeader {
    pub version: u8,
    pub _reserved: [u8; 3],
    pub payload_size: u32,
}
impl FuserResponseHeader {
    pub const VERSION: u8 = 1;
    pub const SIZE: usize = 8;
}
```



5.c.d Application (Binary)

```
let plugin = NativePluginLoader::load_bundle(BUNDLE_PATH)?;
if !plugin.has_binary_transport() {
    return Err(ConsumerError::MissingSymbol(
        "Binary transport not available in this plugin".into(),
    ));
}
let sensor_data = vec![SensorData::new(22.5, 0.3), ... ];
let request = create_request(&sensor_data);
let response = plugin.call_raw(MSG_FUSER_CREATE, &request)?;
let (header, sensor_data) = parse_response(&response)?;
println!(
    "Fuser Response: v{} {:#?} ({} bytes)",
    header.version,
    sensor_data,
```



```
std::mem::size_of_val(sensor_data)
);
```

rustbridge implementation (ix)



```
pub fn create_request(sensor_data: &[SensorData]) -> Vec<u8> {  
    let mut request = Vec::with_capacity(  
        BinaryFuserRequestHeader::SIZE +  
        (sensor_data.len() * std::mem::size_of::<SensorData>()),  
    );  
  
    let header = BinaryFuserRequestHeader::new(  
        (sensor_data.len() * std::mem::size_of::<SensorData>()) as u32  
    );  
    request.extend_from_slice(&header.to_bytes());  
  
    let payload = unsafe {  
        std::slice::from_raw_parts(  
            sensor_data.as_ptr() as *const u8,  
            sensor_data.len() * std::mem::size_of::<SensorData>(),  
        )  
    };  
};
```



```
request.extend_from_slice(payload);  
request  
}
```



```
pub fn parse_response(data: &[u8]) -> Result<(BinaryFuserResponseHeader, &SensorData),
ConsumerError> {
    let header = BinaryFuserResponseHeader::from_bytes(data).ok_or_else(|| {
        ConsumerError::InvalidResponse(format!("Response too small: {} bytes", data.len()))
    })?;

    if header.version != BinaryFuserResponseHeader::VERSION {
        return Err(ConsumerError::InvalidResponse(...));
    }

    let expected_size = BinaryFuserResponseHeader::SIZE + header.payload_size as usize;
    if data.len() < expected_size {
        return Err(ConsumerError::InvalidResponse(...));
    }

    let fuser_data = &data[BinaryFuserResponseHeader::SIZE..expected_size];
    let sensor_data = unsafe { &*(fuser_data.as_ptr() as *const SensorData) };
}
```



```
Ok((*header, sensor_data))  
}
```